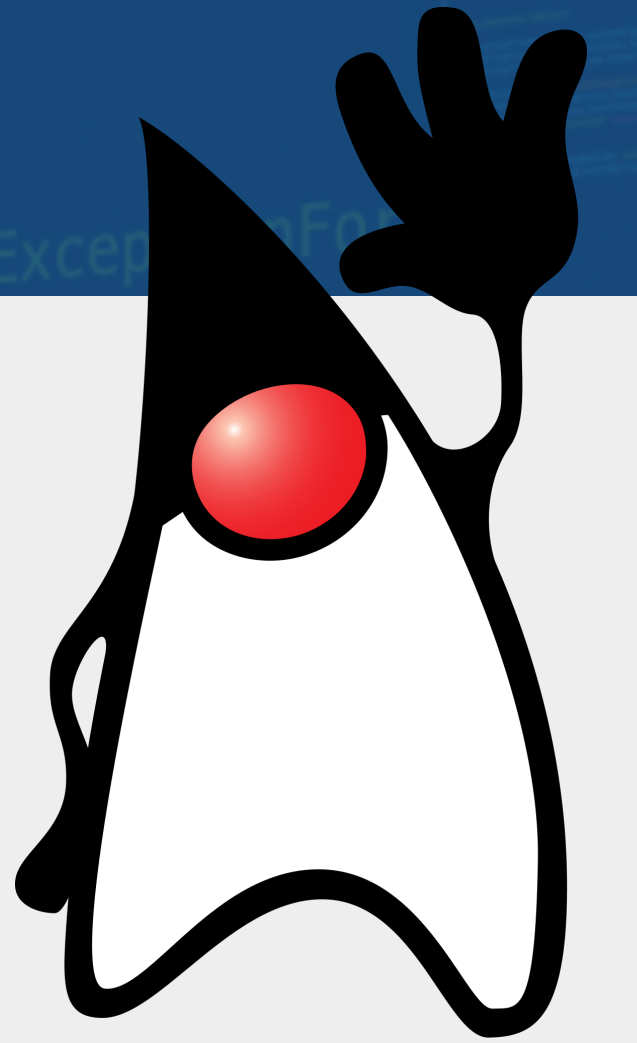


Java Programlama Dili

19. Bölüm Başlatma

Akın Kaldıroğlu



Amaç



- Bu bölümde sınıfı ve nesneyi istenilen duruma getirmek için kullanılan başlatma blokları ele alınacaktır



- **Başlatma**
 - Tanımlama Cümleleri
- **Başlatma Problemi**
- **Başlatma Blokları**
 - final Alanların Başlatılması
- **Başlatma Sırası**
 - Sabit Değişkenlerin Başlatılması
 - Nesnenin Başlatılması
- **Ödevler**
- **Bundan Sonra?**

Başlatma



Üye Değişkenler



- Biliyoruz ki bir nesnenin durumu şu iki tipteki değişkenlerden oluşur:
 - **Nesne değişkenleri (instance (object) variables):** Nesnenin kendisine has durumunu oluşturan değişkenler
 - **Sınıf değişkenleri (class variables):** Nesnelerin ortak durumunu oluşturan, statik değişkenler
- Bunlara **üye değişkenler (member variables)**, **veri üyeleri (data members)** ya da **alanlar (fields)** denir

Başlatma



- **Başlatma (initialization), sınıfın veya oluşturulan nesnenin durumunu kullanıma hazır hale getirmektir**
- Dolayısıyla başlatma, üye değişkenlere; yani sınıf (statik) ve nesne değişkenlerine, ilk değerlerini vermek demektir
- Başlatmada doğal olarak önce sınıf başlatılır sonrasında eğer nesne oluşturuluyorsa nesne başlatılır
- Eğer birden fazla nesne oluşturulursa tabii olarak, birden fazla nesne başlatması söz konusudur

Üye Değişkenlerinin İlk Değeri



- Daha önce üye değişkenlere hiçbir ilk değer atanmadığında, derleyicinin onlara, tipleriyle uyumlu ve en az bilgi sağlayan, varsayılan ilk değer atadığını daha önce öğrenmiştik
- Dolayısıyla üye değişkenler, başlatma sırasında varsayılan ilk değerlere sahip olurlar

boolean	false
byte	0
char	' ' (int olarak 0)
short	0
int	0
long	0 (64 bit)
float	0.0 (32 bit)
double	0.0 (64 bit)
reference	null

InitialValues



- *Nesnenin Durumu* bölümünde `init.InitialValues`
- Bu örnekte tüm üye değişkenlerin varsayılan ilk değerleri incelenmektedir

Başlatma Yöntemleri - I



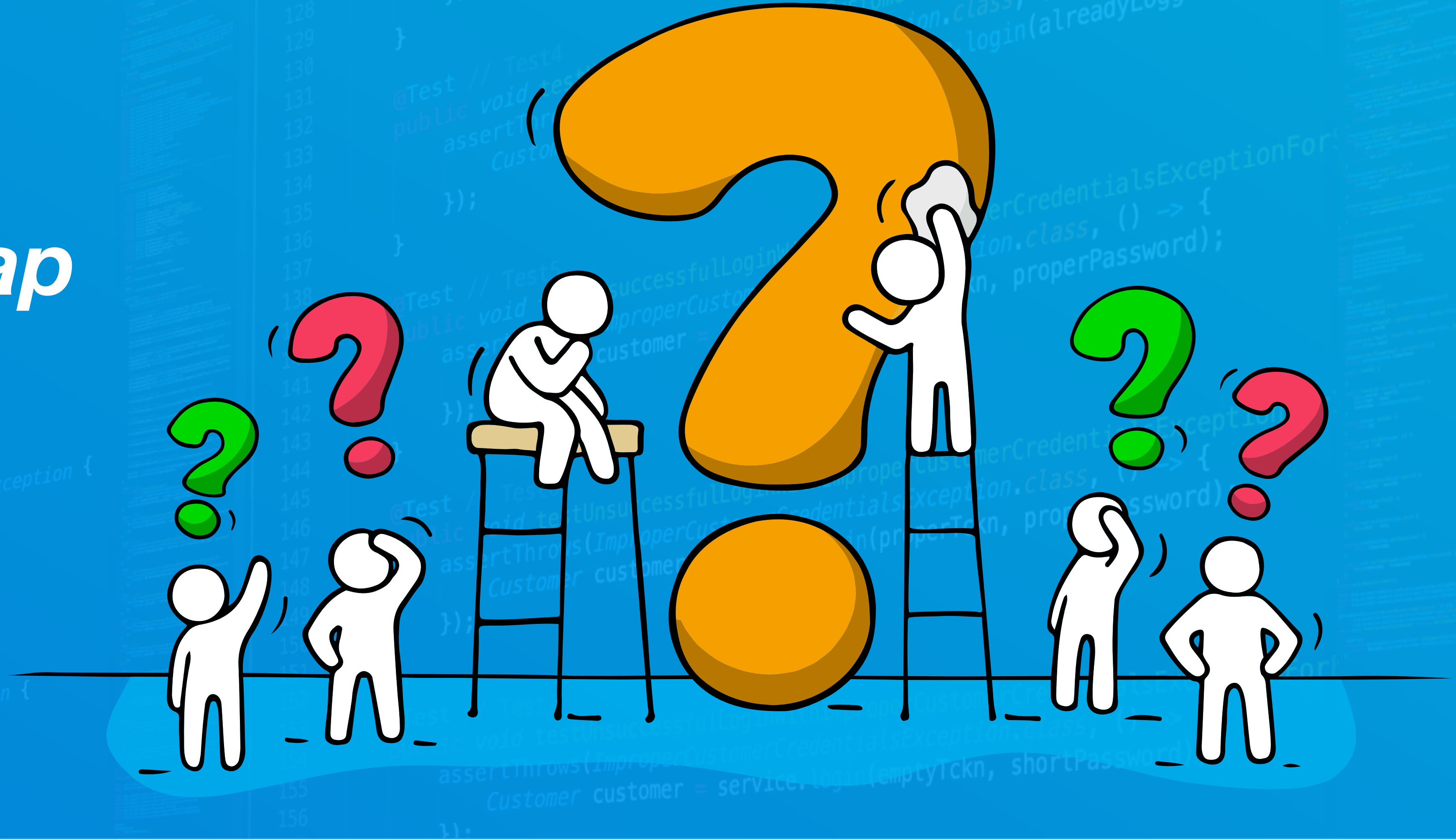
- Java'da üye değişkenleri başlatmak yani ilk değer atamak için 3 yöntem olduğunu daha önce öğrenmiştik:
- **Tanımlama cümleleri (definition statements)**
- **Kurucular (constructors)**
- **Metot çağırısı**

Başlatma Yöntemleri - II



- Daha önce kurucular geniş bir şekilde ele alındı
- Burada, tanımlama cümlesi ve metot çağrısı yoluyla ilk değer atamanın hatırlatılmasını takiben başlatma problemi ve çözüm olarak nesne ve sınıf başlatma blokları ele alınacaktır

Soru ve Cevap Zamanı!





Tanımlama Cümleleri

Tanımlama Cümleleri - I



- Tüm üye değişkenlere, tanıtıldıkları yerde ilk değer verilebilir
- Bu durumda değişkenler için, önce varsayılan sonra da verilen değerle, toplamda iki defa değer ataması söz konusudur
- Böyle ilk değer ataması nesne alanları için çok tercih edilmez, çünkü tüm nesnelerin alanlarında aynı ilk değer söz konusu olur, bu olsa bile her halükarda tekrar atanarak her nesne için değerleri değiştirilecektir
- Ama bu, sınıf alanları için anlamlı olabilir

```
class Employee{  
    int id = 1; // All employees can't have same id.  
    static String nationality = "Türkiye Cumhuriyeti";  
    ...  
}
```

Tanımlama Cümleleri - II



- Tanımlama cümlelerinde birkaç nokta önemlidir:
- İleri referanstan kaçınmak gerekir

```
double totalPercentage = counter * percentage; // Compiler error!  
// Source of error!  
int counter = 1;  
static double percentage = 5.8;
```

- Fakat ileri referans verilen alan **statik** ve başlatılan alan nesne değişkeni ise problem olmayacaktır çünkü sınıfın alanı zaten daha önce başlatılmıştır:

```
// That's ok!  
int counter = 1;  
double totalPercentage = counter * percentage;  
static double percentage = 5.8;
```

Tanımlama Cümleleri - III



- Sınıf değişkeni tanımlarken nesne alanı veya metodu kullanılamaz çünkü nesne alanı ancak nesne yaratıldığında oluşur
- Ama nesne alanı tanımlarken statik alan kullanılabilir

```
int counter = 1;  
static double percentage = 5.8;  
// Compiler error!  
static double totalPercentage = counter * percentage;
```

```
int counter = 1;  
static double percentage = 5.8;  
// That's ok.  
double totalPercentage = counter * percentage;
```

Tanımlama Cümleleri - IV



- Ayrıca tek bir cümlede, aynı tipten birden fazla nesne ya da sınıf değişkeni tanımlanabilir

```
int i = 5, j = 6, k = 7;  
  
static int l = 15, m = 16, n = 17;  
  
int o = 15, static sp = 16; // Compiler error!
```

MemberDefinition



- `MemberDefinition`
- Bu örnekte üye değişkeni tanımları vardır

Metot Çağrısı ile İlk Değer Atama - I



- Metot çağrısıyla ilk değer ataması yapılabilir
- Bu durumda nesne değişkeni için nesne veya sınıf metodu, sınıf değişkeni için ise sadece sınıf metodu kullanılmalıdır
- Unutmayın: Nesne, statik metotlara ulaşabilir ama statik metotlar nesnenin üyelerine ulaşamaz!

```
// Method can be a either static or instance method
boolean isOpen = getDoorStatus();

// getPi() can't be an instance method, it must be a static method
static double pi = getPi();
```

Metot Çağrısı ile İlk Değer Atama - II



- Burada dikkat edilmesi gereken husus, çağrılan metotların henüz gerekli değerler atanmamış üye değişkenleri kullanması durumunda çıkabilecek problemlidir
- Bu durumda ileri referans hatası oluşmaz ama beklenmeyen değerlerle ilk değer ataması yapılabilir

```
// int i = j; // Error! Forward reference
// Not a forward reference but can be misleading!
//j is 0 yet! It is supposed to be initialized to 22
int i = getJ();
int j = 10;

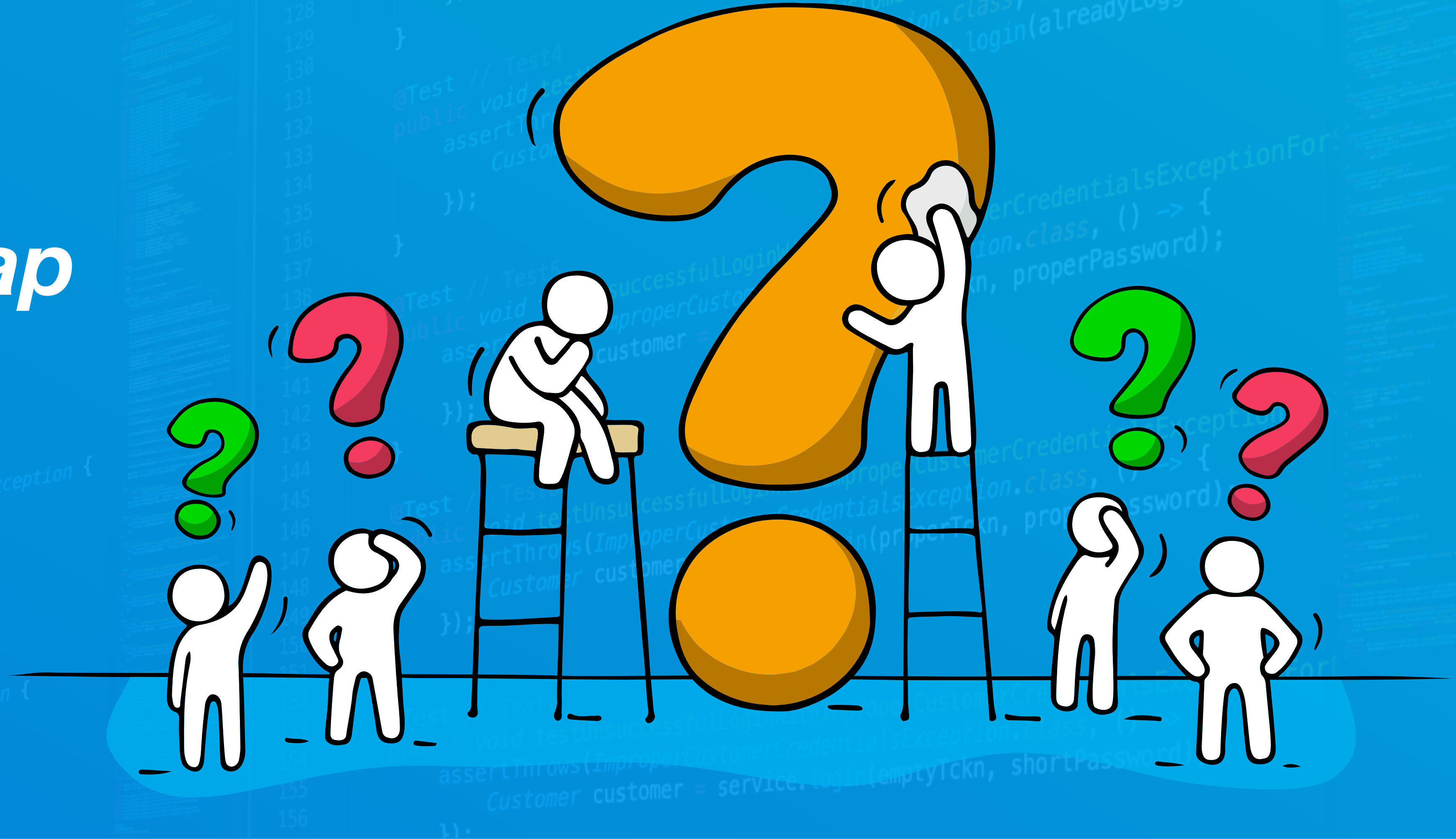
int getJ(){ return 2 * (j + 1);} //2 is returned!
```

InitializerMethods



- `InitializerMethods`
- Bu örnekte ileri referans problemi ile metot çağrısı durumunda aslında ileri referans olacak bir durumun nasıl aşıldığını ama beklenmeyen sonuçlara yol açtığını gözlemleyeceksiniz

Soru ve Cevap Zamanı!



Başlatma Problemi



Bir Problem - I



- Üye değişkenlerinin tanıtlama ve ilk değer atama satırlarını aşağıdaki gibi ayırmak derleme hatasıdır

```
class Class{  
    int i, j = 5, k = 8;  
    i = j * k; // Compiler error!  
  
    static String s;  
    s = "Java"; // Compiler error!  
    ...  
}
```

- İlk değer atama, ya tanıtlama ile beraber olmalıdır ya da kurucu gibi bir alt kapsamda yapılmalıdır

Bir Problem - II



- Neden bu şekilde ayırmak isteyelim ki?
- Eğer atanacak değer, örneğin `for` döngüsüyle bir dizinin elemanlarına ilk değer atanması ya da `if-else`'e ihtiyaç duyan alternatifli durum gibi, uzunca bir kod gerektiriyorsa ya da örneğin sıra dışı durum fırlatıyorsa, bu durumda tanıtmayla atamayı aynı satırda yapamayabiliriz

```
class Class{  
    double coefficient;  
    // LocalDateTime now = LocalDateTime.now();  
    // if (now.isBefore(LocalTime.NOON))  
    //     coefficient = 2;  
    // else  
    //     coefficient = 4;  
    ...  
}
```

Bir Problem - III



- İlk değer atama, tanıtlama ile beraber olmalıdır
- Eğer ayrı olacak ise ilk değer atama muhakkak bir alt kapsamda yapılmalıdır

```
class Class{
    double coefficient;
    // LocalDateTime now = LocalDateTime.now();
    // if (now.isBefore(LocalTime.NOON))
    //     coefficient = 2;
    // else
    //     coefficient = 4;
    ...
}
```

```
class Class{
    double coefficient;
    Class() {
        LocalDateTime now = LocalDateTime.now();
        if (now.isBefore(LocalTime.NOON))
            coefficient = 2;
        else
            coefficient = 4;
    }
}
```

- Alt kapsam için üç alternatif vardır: Kurucular, metotlar ve başlatma blokları

Bir Problem - IV



- Az önce gösterilmişti, metotlar her zaman iyi bir seçenek olmayabilir
- Çünkü metot çağrısı, başlatma sırasını bozar ve istenmeyen durumlara yol açabilir
- Örneğin yandaki durumda kurucuyu kullanmak daha mantıklıdır

```
class Class{  
    double coefficient = initializeCoefficient();  
    double ratio;  
  
    Class(double ratio) {  
        this.ratio = ratio;  
    }  
  
    double initializeCoefficient() {  
        LocalDateTime now = LocalDateTime.now();  
        if (now.isBefore(LocalDateTime.NOON))  
            coefficient = 2 * ratio; // ratio is 0.0  
        else  
            coefficient = 4 * ratio; // ratio is 0.0  
        return coefficient;  
    }  
}
```

Bir Problem - V



- Nesne değişkenlerinin ilk değer ataması kurucularda yapılabilir ama bunun problemi, atama kodların her kurucuya eklenmeyi gerektirmesidir
- Bu durumda copy-paste, yani tekrar eden kodlar söz konusu olacaktır

```
class Class{
    double coefficient;
    double ratio;

    Class() {
        ratio = getFromFile();
        LocalDateTime now = LocalDateTime.now();
        if (now.isBefore(LocalTime.NOON))
            coefficient = 2 * ratio;
        else
            coefficient = 4 * ratio;
    }

    Class(double ratio) {
        this.ratio = ratio;
        LocalDateTime now = LocalDateTime.now();
        if (now.isBefore(LocalTime.NOON))
            coefficient = 2 * ratio;
        else
            coefficient = 4 * ratio;
    }
}
```

Bir Problem - VI



- Bundan kaçınmak için kurucular birbirlerini `this ()` ile çağırabilir
- Ama bu da her zaman, özellikle karmaşık dolayısıyla pek çok kurucunun olduğu durumlarda, yapılamayabilir ve yine copy-paste'e sebep olur
- Benzer durum farklı ilk değer atama yöntemlerinde de söz konusudur

```
class Class{
    double coefficient;
    double ratio;

    // Can't use this() call in this case!
    Class() {
        ratio = getFromFile();
        LocalDateTime now = LocalDateTime.now();
        if (now.isBefore(LocalDateTime.MIDNIGHT))
            coefficient = 2 * (ratio + 1);
        else
            coefficient = 4 * (ratio - 1);
    }

    Class(double ratio) {
        this.ratio = ratio;
        LocalDateTime now = LocalDateTime.now();
        if (now.isBefore(LocalDateTime.NOON))
            coefficient = 2 * ratio;
        else
            coefficient = 4 * ratio;
    }
}
```

Bir Problem - VII



- Bir diğer yöntem ise ilk değer atamayı metotlarda yapıp, bu metotları kuruculardan çağırmaktır
- Bu durumda bu metotların her kurucuda çağrılması gereklidir ve unutulduğunda hataya sebep olur
- Özellikle sonradan eklenen kurucularda bu metotların çağrılması unutulabilir

Bir Problem - VIII



- Konu sınıf değişkenleri için daha da zordur
- Sınıf değişkenlerinin basit değer atamasıyla yapılamayacak kadar karmaşık olan ilk değer atamaları nerede olabilir?
- Kurucuda olamaz çünkü nesne oluşturulmayabilir, oluşturulsa bile statik başlatmaların, nesne oluşturulmadan önce bitmiş olması gereklidir

Bir Problem - IX



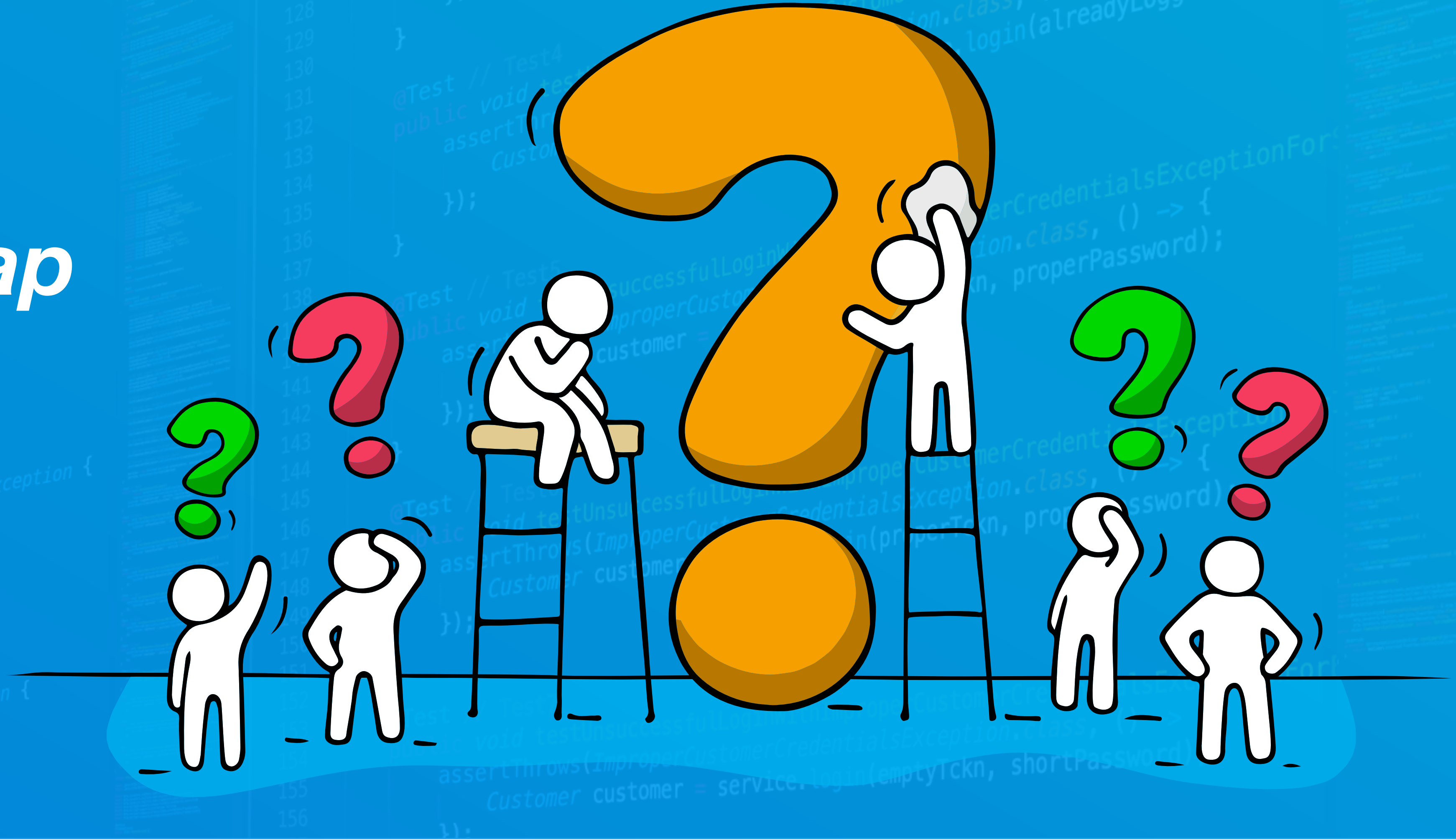
- Bunun çözümü statik değişkenlerin başlatmasını yapan bir statik metot olabilir
- Sınıf ile ilgili yapılacak ilk şey bu metodu çağırmaktır
- Bu yöntem az önce gösterilmişti, iyi bir seçenek değildir

InitializationProblem1, 2, 3 ve 4



- `problem.InitializationProblem1`
- `problem.InitializationProblem2`
- `problem.InitializationProblem3`
- `problem.InitializationProblem4`
- Bu örneklerdeki ilk değer atamaların nasıl yönetildiğini gözlemleyin

Soru ve Cevap Zamanı!



Başlatma Blokları



Başlatma Blokları - I



- Tüm bu ilk değer atama problemlerinin çözümü **başlatma bloğudur (initializer block)**
- Bir üye değişkene atanacak ilk değer için örneğin hesap yapan uzunca kod söz konusuysa hatta bu hesapta sıra dışı durum fırlatılması ihtimali de varsa, dolayısıyla yönetilmesi gerekliyse, yukarıdaki problemlerden kaçınmak için en uygun çözüm başlatma bloklarını kullanmaktır
- Başlatma blokları, tanımlama cümlesi, kurucu ve metot çağrısı ile başlatmaya çok iyi bir alternatiftir

Başlatma Blokları - II



- Başlatma bloklarının, sınıf ve nesne için, statik olan ve olmayan değişkenlere yönelik olmak üzere iki türü vardır
- Başlatma blokları genel olarak sınıfın başında herhangi bir alt bloğun dışında ve tabii olarak **{ }** ile tanımlanır
- Sınıf başlatma blokları **static** ile nitelendirilir
- Nesne başlatma blokları için bir niteleyiciye gerek yoktur

```
class Car{  
    int speed;  
    {  
        speed = ...;  
    }  
  
    static int carCount;  
    static {  
        carCount = ...;  
    }  
    ...  
}
```

InitializationSolution



- `problem.InitializationSolution`
- `InitializationProblem1`, `InitializationProblem2`, `InitializationProblem3` ve `InitializationProblem4`'de var olan problemlerin çözümü bu örnekte verilmektedir

Sınıf Başlatma Blokları - I



- Sınıf değişkenlerinin basit değer atamasıyla yapılamayacak kadar karmaşık olan ilk değer atamaları statik başlatma bloklarında yapılmalıdır
- Statik üyelerin başlatılması için oluşturulmuş olan statik başlatma bloklarında sadece statik üyeler kullanılabilir, nesne üyeleri kullanılamaz çünkü onlar için nesneye ihtiyaç vardır
- Dolayısıyla static başlatma bloklarında sadece statik değişkenler kullanılabilir ve statik metot çağrıları yapılabilir

Sınıf Başlatma Blokları - II



- Sınıf başlatma blokları, sınıf yüklendikten sonra başlatılırken çalıştırılır
 - Dolayısıyla sınıf başlatma blokları sadece bir kere çalışır
- Birden fazla sınıf başlatma bloğu varsa, yukarıdan aşağıya fiziksel sıra ile çalıştırılır

Nesne Başlatma Blokları - I



- Kurucular ve nesne başlatma blokları, nesne değişkenlerine ilk değer atamak için ideal yerlerdir
- Yukarıda bahsedilen kod tekrarı gibi problemlerden dolayı, özellikle basit değer atamaları dışında, daha karmaşık kod gerektiren atamalar için nesne başlatma blokları, kuruculara tercih edilmelidir
- Nesne başlatma blokları her nesne oluşturulmasında tekrar çalıştırılır
 - Çünkü nesne değişkenleri, nesneye hastır

Nesne Başlatma Blokları - II



- Nesne başlatma blokları derleyici tarafından `class` dosyasında, içinde `this ()` olmayan tüm kuruculara kopyalanır
- Dolayısıyla nesne başlatma blokları, daima sınıf başlatma bloklarından sonra ve sadece nesne oluşturuluyorsa çalıştırılır; aksi takdirde çalıştırılmaz
- Birden fazla nesne başlatma bloğu varsa bloklar yukarıdan aşağıya fiziksel sıra ile çalışır

Başlatma Blokları - IV



- Başlatma blokları, aslında bir yerel kapsam (local scope) oluşturan, metot benzeri bloklardır
- Metotlardan farkı sadece JVM tarafından çalıştırılabilirler, argüman almaz ve değer döndürmezler
- Başlatma bloklarında hiçbir şekilde **return** kullanılamaz
- Başlatma blokları metot çağrısı yapabilir
- Başlatma bloklarında alt bloklar tanımlanabilir

Başlatma Blokları - V



- Bir değişkene birden fazla başlatma bloğunda tekrar tekrar değer ataması yapılabilir
- Başlatma bloğu, üye değişkenle aynı isimde yerel değişken tanımlayabilir, bu durumda üye değişkene ulaşmak için sınıfın ismi ya da `this` kullanılır

InitializerBlocks



- `InitializerBlocks`
- Bu örneği `main` metodu şu şekillerde çalıştırın:
 - tamamen boşken
 - bir `InitializerBlocks` nesnesi yaratan kod ile
 - İki `InitializerBlocks` nesnesi yaratan kod ile

Başlatma Blokları - VI



- Başlatma blokları ileriye referans veremez
- Üye değişken olarak tanıtılan diziye, başlatma bloğunda, { } içinde sıralı değer vererek, elemanlarına değer atanamaz
- Ayrıca başlatma blokları üye (member) değildir
- Nesne başlatma blokları checked exception fırlatabilir ama statik başlatma blokları fırlatamaz
- Sıra dışı durum yönetimi, sonraki eğitimde ele alınacaktır

ForwardReferences



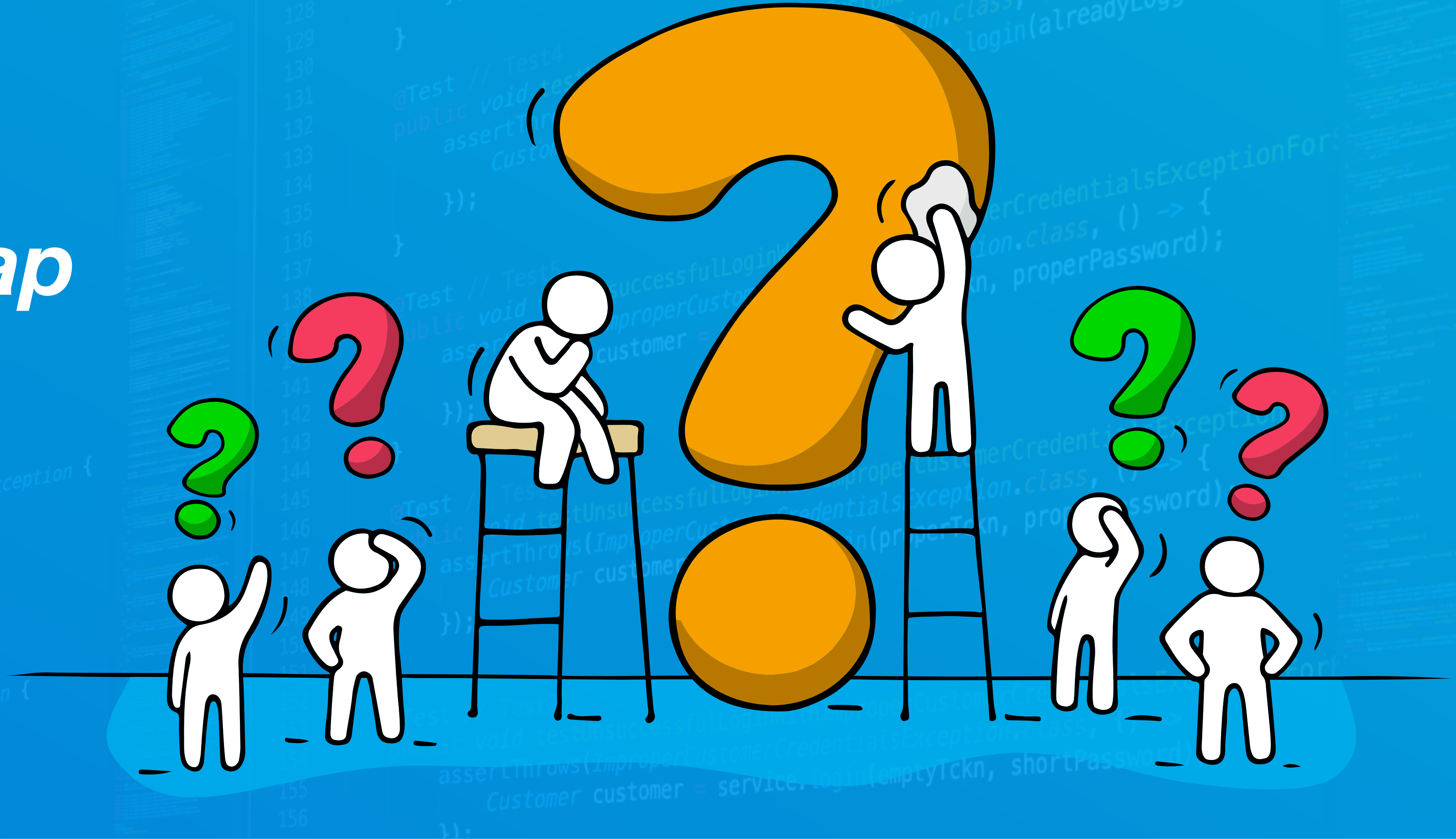
- `problem.ForwardReferences`
- Bu örnekte başlatma bloklarında yapılan doğru ve yanlış olan ileri referanslara örnekler verilmiştir

InitializerBlockProblems



- `problem.InitializerBlockProblems`
- Bu örnekte başlatma bloklarıyla ilgili dikkat edilmesi gereken birkaç nokta ele alınmıştır

Soru ve Cevap Zamanı!





final Alanların Başlatılması

final Alanların Başlatılması



- `final` nesne değişkenleri eğer boş olarak tanımlanırlarsa, ya bir nesne başlatma bloğunda ya da içinde `this ()` olmayan tüm kurucularda ilk değer atamaları (initialization) yapılmalıdır, aksi takdirde derleme hatası alınır
- `final` sınıf (`static`) değişkenleri eğer boş olarak tanımlanırlarsa, bir `static` başlatma bloğunda ilk değer atamaları (initialization) yapılmalıdır aksi takdirde derleme hatası alınır

FinalInitialization



- `FinalInitialization`



Ödev



- Bir önceki örnekte, `FinalInitialization` sınıfında, `final` bir değişken olan `i` değişkenine, nesne başlatma bloğunda ya da içinde `this ()` olmayan kurucularda değer ataması yapılmadığında derleyicinin `Variable i might not have been initialized` mesajıyla hata vermesini nasıl açıklarsınız?
- Kodu komut satırında derlediğinizde derleyicinin hata satırlarını gösterdiğini fark edeceksiniz; bu size yardımcı olabilir

MightNotHaveBeenInitialized



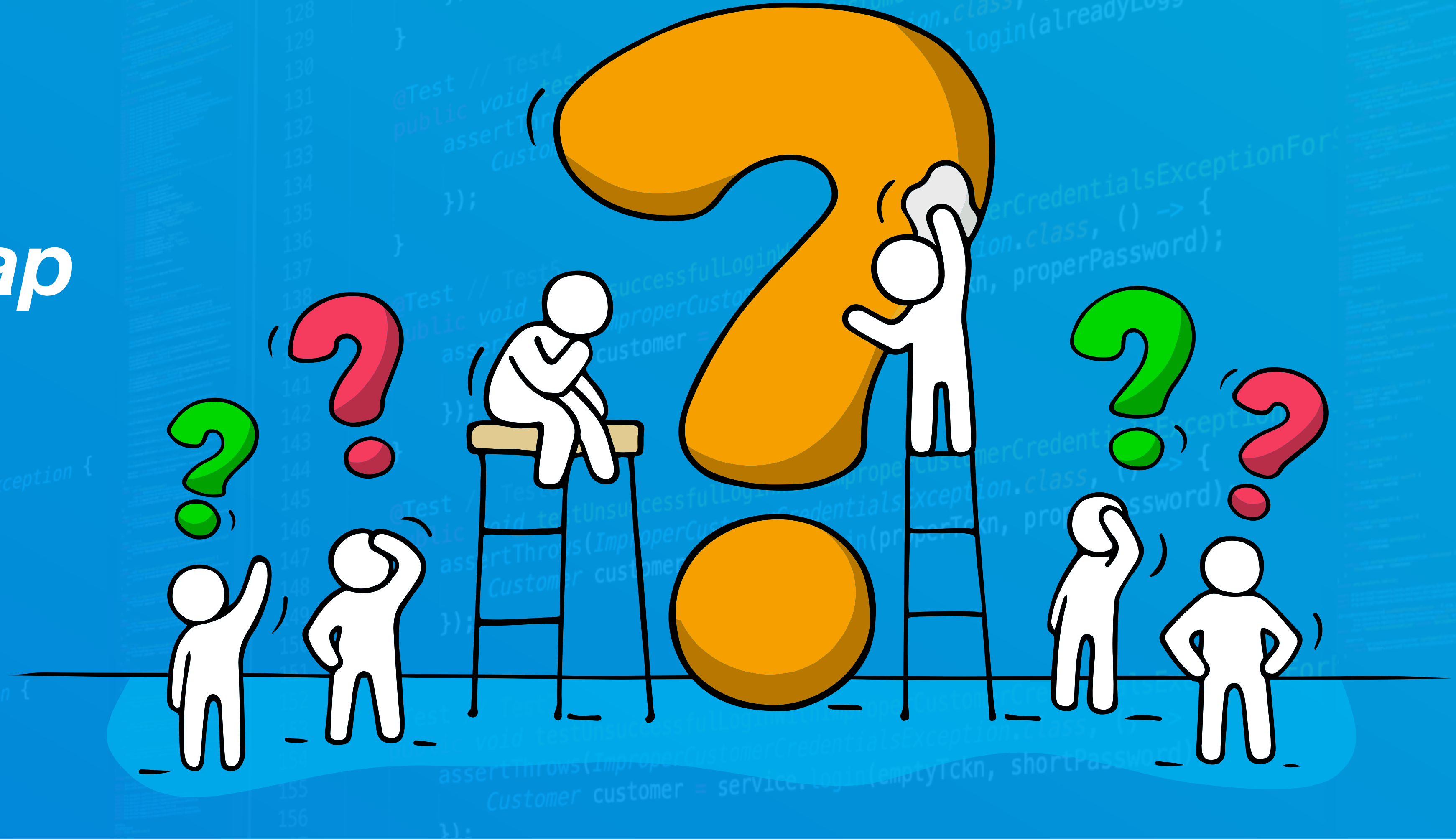
- `error.MightNotHaveBeenInitialized`

MightAlreadyHaveBeenAssigned



- `error.MightAlreadyHaveBeenAssigned`

Soru ve Cevap Zamanı!



Başlatma Sırası



Başlatma Sırası



- **Başlatma sırası (initialization order)**, sınıfın yüklenmesi ve sonra nesnelerinin oluşturulmasında, değişkenlerin başlatılmasıyla başlatma blokları ve kurucu çağrılarının hangi sıra ile yapıldığını belirler
- Burada verilecek olan başlatma sırası, ilerideki derslerde miras (inheritance) gibi yapılarla daha da genişletilecektir

Sınıf Yükleme (Class Loading)



- JVM, bir sınıfa ihtiyaç duyduğunda onu **sınıf yükleyici (class loader)** olan `java.lang.ClassLoader` sınıfının bir nesnesi ile yükler
- Yüklenen sınıfın `class` dosyası belleğe alınır ve onunla `java.lang.Class` sınıfının bir nesnesi üretilir
- Sonra sınıfta kullanılan dolayısıyla bağımlı olunan (dependency) diğer sınıf vb. yapılarla ilişkiler kurulur (**linking**) ve farklı kontroller (**verification**) yapılır

Sınıfı Hazırlama (Preparing Class)



- JVM, sonrasında **sınıfı hazırlamaya (preparing class)** geçer
- Hazırlama sırasında sınıf değişkenleri ve sabitelere varsayılan değerler atanır
- Ve bu değerler gözlemlenebilir (observable): Atanan bu değerlere, başlatma blokları ve kurucularda ulaşmak mümkündür
- Hazırlamada herhangi bir kod çalıştırılmaz, sadece sınıfın değişkenleri ve sabiteleri varsayılan değerlerle başlatılır (initialize)

Sınıf Başlatma (Class Initialization) - I



- Sonrasında **sınıf başlatma (class initialization)** gerçekleşir:
 - Bu safhada tüm statik alanlar, atamalarla kodda verilen ilk değerlerine eşitlenir
 - Sonra da statik başlatma blokları çalıştırılır
 - Birden fazla statik alan ve başlatma bloğu varsa, başlatma yukarıdan aşağıya doğru ilerler

Sınıf Başlatma (Class Initialization) - II



- Şunlar olmadan önce sınıfın başlatılmış olması gereklidir:
 - Sınıfın bir nesnesinin oluşturulması
 - Sınıfın statik bir metodunun çağırılması
 - Sınıfın statik bir alanına atama yapılması
 - Sınıfın *sabit değişken olmayan* bir statik bir alanının kullanılması

Sınıf Başlatma (Class Initialization) - III



- Dolayısıyla sınıfın statik bir üyesine ulaşmak ya da nesnesini oluşturmak yani kurucusunu çağırmak, sınıfın yüklenmesini, hazırlanıp başlatılmasını sağlar
- Ama statik üye, sabit değişken ise bu durumda sınıf başlatılmaz, sadece sabit değişken olan üye başlatılır

Sınıf Başlatma (Class Initialization) - IV



- Aslen derleyici, tüm statik alan atamalarıyla statik başlatma bloklarını `class` dosyasında tek bir statik başlatma bloğu altına toplar
- Dolayısıyla sınıfın en başında sadece statik alan tanıtları kalır, statik alanlara kodda yapılan tüm ilk değer atamaları `class` dosyasındaki bir tane olan statik başlatma bloğunda yapılır

Sınıf Başlatma (Class Initialization) - V



- Bu toplamada statik alan ve başlatma bloklarının fiziksel sırasına dikkat edilir
- Sınıf sadece bir kez, daha önce bahsedilen 4 şey yapılıyorsa, onlardan önce yüklenir, hazırlanır ve başlatılır

ClassInitializationOrder1



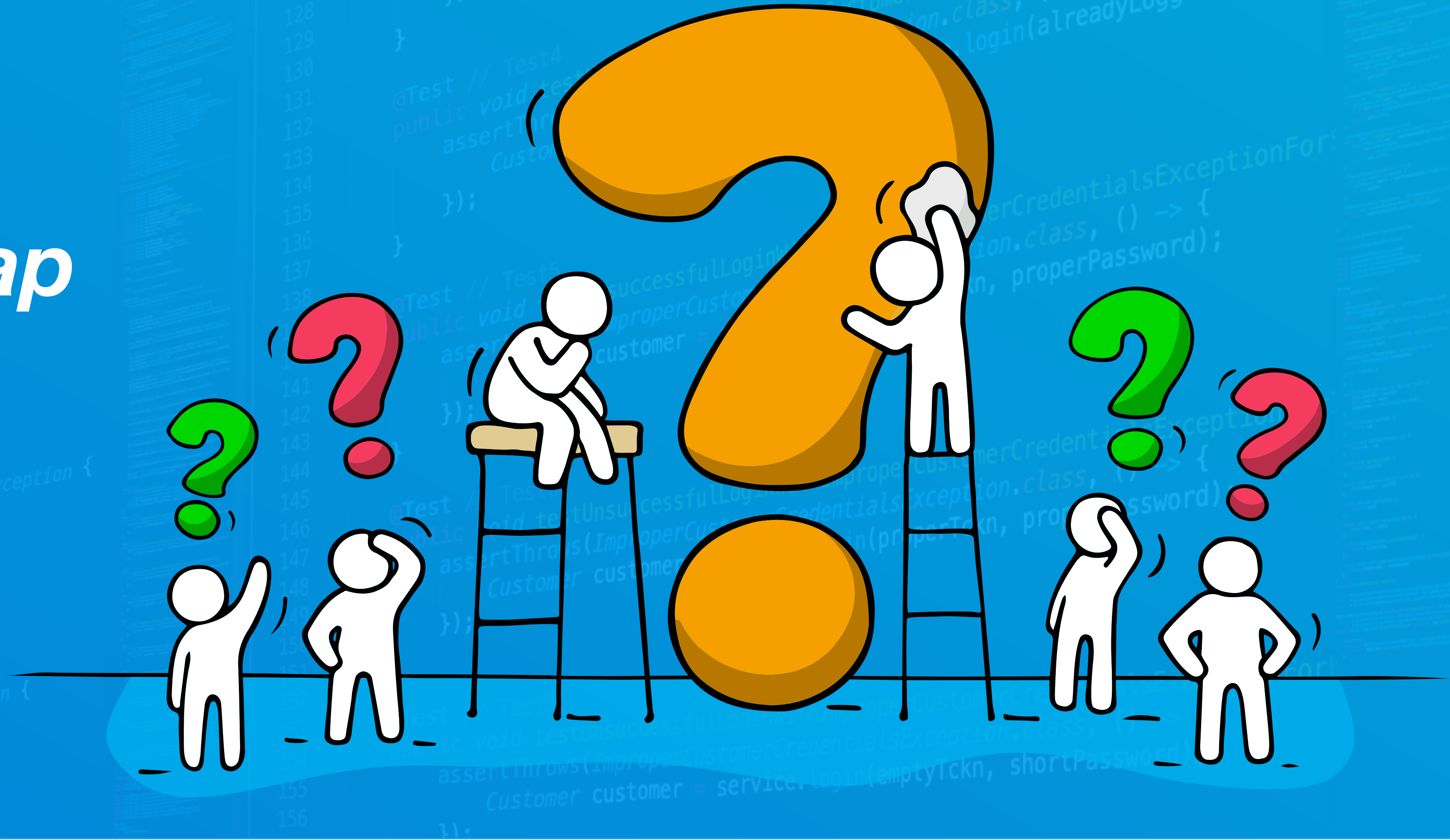
- `order.ClassInitializationOrder1`
- Bu örneği sırasıyla `main` metodun aşağıdaki halleri ile çalıştırın:
 - Tamamen boş
 - Sadece `ClassA` a ile
 - Sadece `System.out.println(ClassA.i)` ile

ClassInitializationOrder1



- Sadece `ClassA.printInfo()` ile
- Hem `System.out.println(ClassA.i)` hem de `ClassA.printInfo()` ile ve
- Sadece `new ClassA()` ile

Soru ve Cevap Zamanı!





Sabit Değişkenlerin Başlatılması

Sabit Değişken (Constant Variable) - I



- Daha önce “Birden fazla statik alan ve başlatma bloğu varsa, başlatma yukarıdan aşağıya doğru ilerler.” dendi
- Bunun tek istisnası, statik olan sabit değişkenlerin (constant variable) başlatılmasıdır
- Statik olan sabiteler ya da sabit değişkenler diğer statik değişkenlerden önce başlatılır

Sabit Değişken (Constant Variable) - II



- Sabit değişken, **final** olan ilkel ya da **String** alanlardır ve derleme zamanında belirlenebilen sabit değerleri vardır
- Sabit değişken, boş (blank) olarak tanıtilamaz
- Sabit değişken ve sabit ifade (constant expression) daha önce genişçe ele alınmıştı

Sabit Değişken (Constant Variable) - III



- Diğer tüm üye değişkenlerin derleyici tarafından atanan varsayılan değerleri gözlemlenebilir olduğunu da daha önce belirttik
- Örneğin kod ile değer atamadan önce başlatma bloğunda ya da kurucuda değeri alınıp başka bir değişkene atanabilir ya da yazdırılabilir
- Ama sabit değişkenin varsayılan ilk değerleri gözlemlenemez çünkü sabit değişken, boş (blank) olarak tanıtılamaz, değerine ulaşıldığında zaten başlatılmış olması gerekir, aksi takdirde derleme hatası alınır

Sabit Değişken (Constant Variable) - IV



- Eğer bir sınıfın sadece sabit değişkenlerine ulaşılıyorsa bu durumda statik bloklar başlatma sırasında çalışmayabilir
- Yani eğer `static` ve `final` olan bir değişkenin değeri derleme zamanında belirlenebiliyorsa, bu durumda belki de istenen sadece bu değere ulaşmaktır
- Dolayısıyla, JVM sınıfın gerektiği kadarını, sadece sabiteleri başlatır, geri kalanı başlatmaz
- Aslolan sınıfın (ve tabii sonrasında nesnelerin) durumlarını (state) doğru bir şekilde tesis etmektir

ClassInitializationOrder2



- `order.ClassInitializationOrder2`
- Bu örnekte `final` ve sabit değişken olan sınıf üyelerinin davranışı gözlemlenecektir
- `i`, `j` ve `s` `static` değişkenlerini `final` yaparak ve yapmadan aşağıdaki satırları çalıştırın:

ClassInitializationOrder2



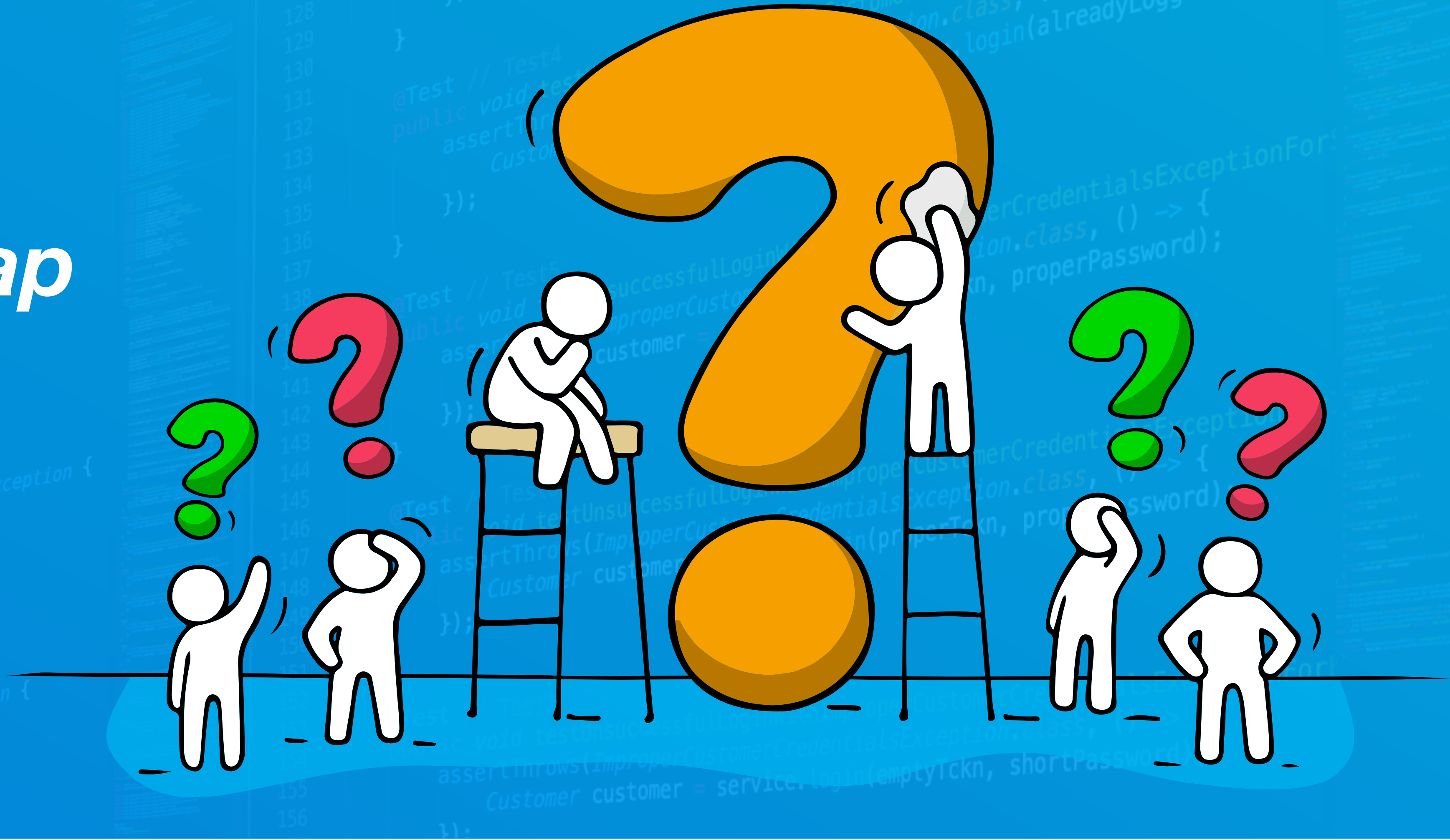
```
System.out.println(ClassAA.i);
```

```
System.out.println(ClassAA.j);
```

```
System.out.println(ClassAA.s);
```

En son olarak `static` ve `final` `i`, `j` ya da `s`'den birine değer atamayarak sabite olmaktan çıkarın ve tekrar çalıştırın

Soru ve Cevap Zamanı!





Nesnenin Başlatılması

Nesne Oluşturma - I



- Java derleyicisi, derleme ile `class` dosyası üretirken, ister tanıtımda olsun ister nesne başlatma bloklarında olsun, tüm nesne değişkeni atamalarını, içinde `this()` çağrısı olmayan tüm kuruculara kopyalar
- Sınıf başlatma sırasında, sınıf değişkenlerine yapılan tüm atamalar da benzer şekilde tek bir sınıf başlatma bloğuna toplanıyordu

Nesne Oluşturma - II



- Bu şekilde, alan başlatmalarının bir kurucuda ilk yapılan iş olması sağlanır
- İçinde `this ()` çağrısı olan kurucular, nesne değişkenlerinin başlatılmalarını öncelikli olarak yapamayacaklarından, başlatma kodları sadece içinde `this ()` olmayan kuruculara kopyalanır
- Kopyalama dolayısıyla da başlatma, kaynak koddaki fiziksel sıraya göre yapılır

Nesne Oluşturma - III



- Eğer sınıfın bir nesnesi oluşturuluyorsa sırayla şunlar yapılır:
 - Önce heapte nesne ve değişkenleri için yer ayrılır
 - Daha sonra nesne değişkenlerine varsayılan değerleri atanır
 - Sonra nesne oluşturma ifadesinde belirtilen kurucu çağrısı yapılır
 - Nesne başlatma blokları, içinde **this (...)** olmayan kuruculara kopyalanmıştır!

Nesne Oluşturma - IV



- Eğer çağrılan kurucu içinde **this (...)** ile diğer bir kurucuya dallanma varsa, o kurucu çağrılır ve bu şekilde çağrılarla, içinde **this (...)** olmayan en son kurucuya kadar dallanma devam eder
- Sonra nesne değişkenlerine yapılan atamalar ve nesne başlatma blokları çalıştırılır, bunlar birden fazla ise yukarıdan aşağıya doğru fiziksel sıra ile çalıştırılır
- Sonra içinde bulunan kurucunun geri kalanı çalıştırılır

Nesne Oluşturma - V



- İçinde `this (...)` olmayan en son kurucu bittikten sonra geriye doğru kurucuların çağrıları tamamlanır ve akış, ilk çağrılan kurucuya geri döner
- Sonra da ilk çağrılan kurucu tamamlanır
- Eğer kuruculara nesne alanlarına atanmak üzere argüman geçiliyorsa, kurucularda bu atamalar da yapılır

Nesne Oluşturma - VI



- Bu işlemlerden sonra oluşturulan ve durumu belirlenen nesneyi gösteren referans geri döndürülür
- Ve her nesne oluşturulduğunda bu işlemler tekrarlanır

ObjectConstruction



- `order.ObjectConstruction`
- Bu örneği sırasıyla `main` metodun şu farklı halleri ile çalıştırın:
 - Tamamen boş
 - Sadece `ClassB b` ile
 - Sadece `ClassB b1 = new ClassB()` ile ve

ObjectConstruction



- `ClassB b1 = new ClassB()`
`ClassB b2 = new ClassB()` ile ve
- Sadece `new ClassB().printInfo()`

Başlatma Sırası - I



- Dolayısıyla başlatma sırası şu şekildedir:
 - Sınıf değişkenleri (statik başlatma blokları dahil)
 - Nesne oluşturuluyorsa, her nesne için
 - Nesne değişkenleri (nesne başlatma blokları dahil)
 - Kurucu çağrısı ve varsa **this ()** ile yapılan tüm kurucu çağrıları

Başlatma Sırası - II



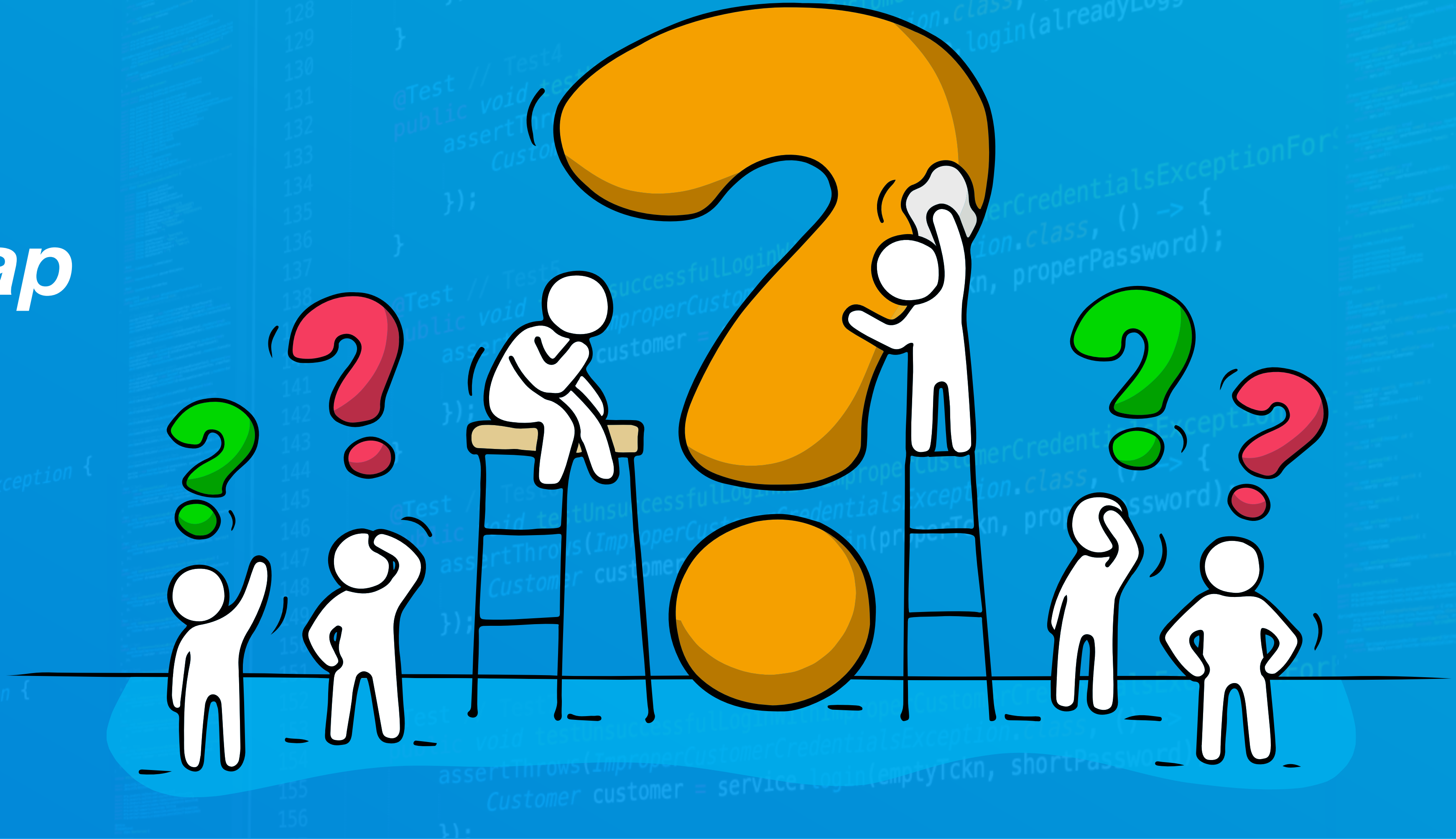
- Birden fazla sınıf ve nesne değişkeni varsa başlatma sırası, fiziksel sırayla belirlenir, önce gelen önce başlatılır
- Eğer üye değişkenler de referans değişkeniyse, onların da başlatılmaları gereklidir
- Bu şekilde bağımlılıklar (dependency) üzerinden giderek gerekli tüm sınıflar ve nesneler başlatılır
- Bu şekilde başlatma bir sınıftan ve nesnesinden başlar ve kullanılan pek çok sınıfa ve nesnesine kadar ulaşır

InitializationOrder



- `order.InitializationOrder`

Soru ve Cevap Zamanı!



Ödevler





1. Önceki bir bölümde geçen `university.Test` sınıfını kullanarak sınıfların ve nesnelerin başlatma sırasını tahmin edin
 - Gerekli konsola yazma ifadelerini kurucularda kullanarak başlama sırasını gözlemleyin



2. `Weird` sınıfındaki `main` metodu çalıştırın ve çalışmasını açıklayın

Bundan Sonra?



Bundan Sonra?



- Bundan sonra bellek yönetimini inceleyeceğiz

Bölüm Sonu

Soru ve Cevap Zamanı!

